

CSC1031

OOP

```
public Product(String productName, long price, boolean inStock, List<String> tags) {  
    this.productName = productName != null ? productName : "Unknown";  
    this.price = Math.max(price, 0);  
    this.inStock = inStock;  
    this.tags = tags != null ? new ArrayList<>(tags) : new ArrayList<>>();  
}
```

// Chain constructors to the main constructor

```
public Product(String productName, long price, boolean inStock) {  
    this(productName, price, inStock, new ArrayList<>());  
}
```

```
public Product(String productName, long price) {  
    this(productName, price, false);  
}
```

```
public Product(String productName) {  
    this(productName, 0, false);  
}
```

```
public Product() {  
    this("Unknown");  
}
```

```
public Product(String productName, long price, boolean inStock, List<String> tags) {  
    this.productName = productName != null ? productName : "Unknown";  
    this.price = Math.max(price, 0);  
    this.inStock = inStock;  
    this.tags = tags != null ? new ArrayList<>(tags) : new ArrayList<>();  
}  
  
// Chain constructors to the main constructor  
public Product(String productName, long price, boolean inStock) {  
    this(productName, price, inStock, new ArrayList<>());  
}  
  
public Product(String productName, long price) {  
    this(productName, price, false);  
}  
  
public Product(String productName) {  
    this(productName, 0, false);  
}  
  
public Product() {  
    this("Unknown");  
}
```

```
graph TD  
    C1["this('Unknown')"] --> C2["this(productName, 0, false)"]  
    C2 --> C3["this(productName, price, false)"]  
    C3 --> C4["this(productName, price, inStock, new ArrayList<>())"]  
    C4 --> C5["this.tags = tags != null ? new ArrayList<>(tags) : new ArrayList<>();"]
```

What is the Ternary Operator?

```
a = productName != null ? productName : "Unknown";
```

A compact alternative to an if-else statement.

```
condition ? expression_if_true : expression_if_false;
```

Also known as the **conditional operator**.

Example

```
int num = 10;  
String result;  
if (num > 0) {  
    result = "Positive";  
} else {  
    result = "Non-positive";  
}
```

```
String result = (num > 0) ? "Positive" : "Non-positive";
```

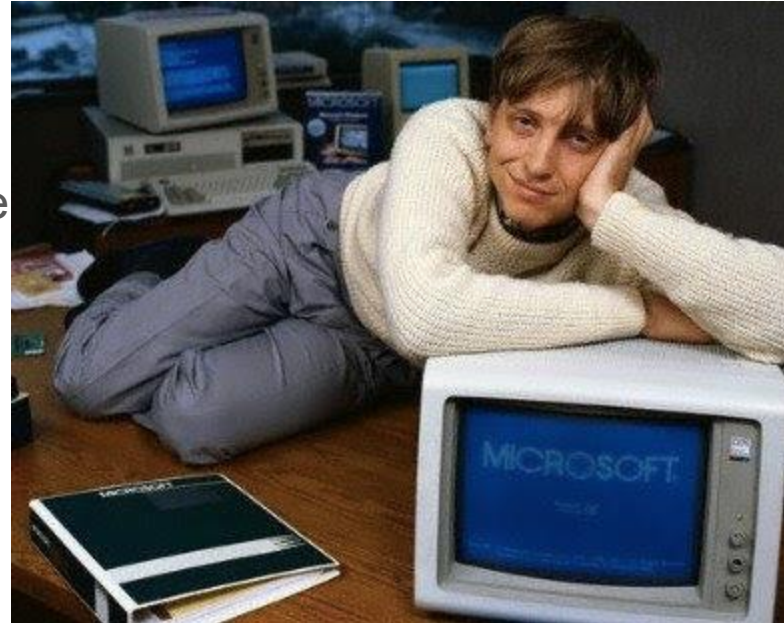
Syntax Breakdown

`variable = (condition) ? value_if_true : value_if_false;`

condition → A boolean expression (true or false).

value_if_true → Executed if the condition is true.

value_if_false → Executed if the condition is false



For

```
for (Player player : players) {  
    ...  
}
```

This is an **enhanced for-loop** (also called a "for-each" loop).

It is used to iterate over collections like `ArrayList`, `HashSet`, or arrays.

It eliminates the need for a traditional `for` loop with an index.

Key Components

for (**Player** **player** : **players**) {

1. **Player** **player** → A variable that holds the current element in the loop.
2. **players** → A collection
 - a. (e.g., **List<Player> players = new ArrayList<>();**).
3. **:** → Separates the variable from the collection, meaning "for each element in."

```
for (int i = 0; i < players.size(); i++) {  
    Player player = players[i]  
    // Process the player  
}
```


Example

```
String[] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
for (String abc : cars) {  
    System.out.println(abc);  
}
```



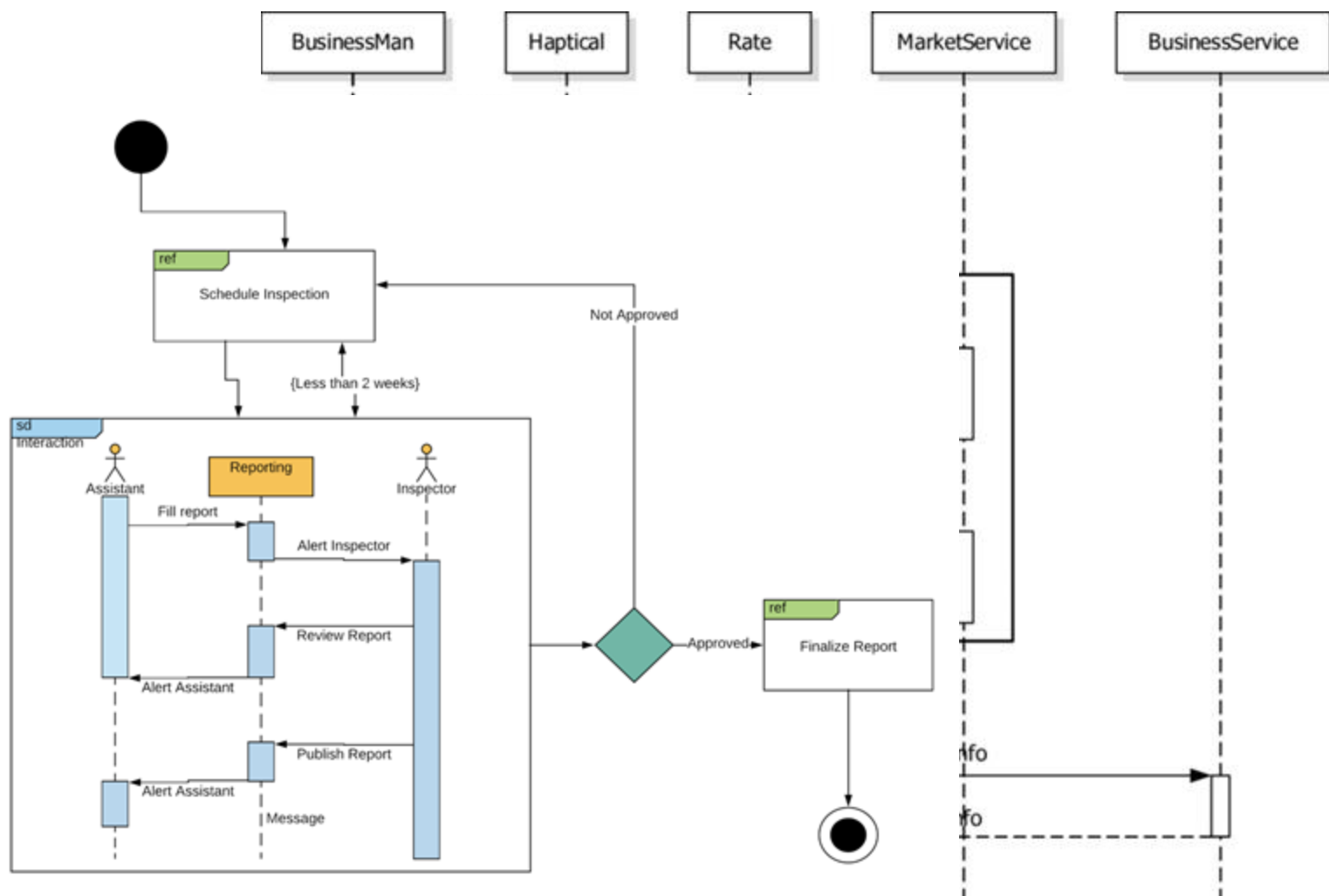
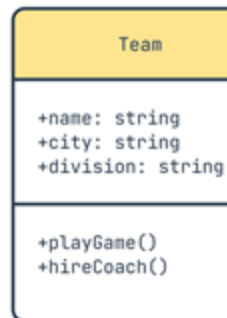
```
for (var player : players) {  
    System.out.println(player.name);  
}
```



UML

- Unified Modeling Language (UML) is a standard for modeling object-oriented systems.
- Helps in planning, visualizing, and documenting software architecture.
- Commonly used in system design, software documentation, and team collaboration.





Types of UML Diagrams

Structural Diagrams: Class Diagram, Object Diagram, Component Diagram, Deployment Diagram.

Behavioral Diagrams: Use Case Diagram, Sequence Diagram, Activity Diagram, State Diagram.

Focus today: Class Diagrams for OOP.

Class diagrams

A class diagram is a diagram used in designing and modeling software to describe classes and their relationships.

- High level of abstraction
- No source code
- The diagram shows the **names** and **attributes** of the classes, **connections** between the classes, and sometimes also the **methods** of the classes.

Describing class and class attributes

class attributes are written

"attributeName: attributeType"

+ public

- private

protected

```
public class Person {  
    private String name;  
    private int age;  
}
```

Person
-name:String
-age:int

Describing class constructor

The parameters are written the same way class attributes are:

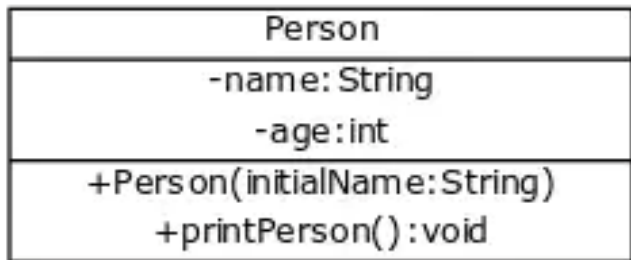
"parameterName: parameterType".

```
public class Person {  
    private String name;  
    private int age;  
  
    public Person(String initialName) {  
        this.name = initialName;  
        this.age = 0;  
    }  
}
```

Person
-name: String -age: int
+Person(initialName: String)

Describing class methods

```
public class Person {  
    private String name;  
    private int age;  
  
    public Person(String initialName) {  
        this.name = initialName;  
        this.age = 0;  
    }  
  
    public void printPerson() {  
        System.out.println(this.name + ", age " + this.age + " years");  
    }  
}
```



constructors are listed first

We also write the **return type of a method** in the class diagram.

```
public class Person {  
    private String name;  
    private int age;  
  
    public Person(String initialName) {  
        this.name = initialName;  
        this.age = 0;  
    }  
  
    public void printPerson() {  
        System.out.println(this.name + ", age " + this.age + " years");  
    }  
  
    public String getName() {  
        return this.name;  
    }  
}
```

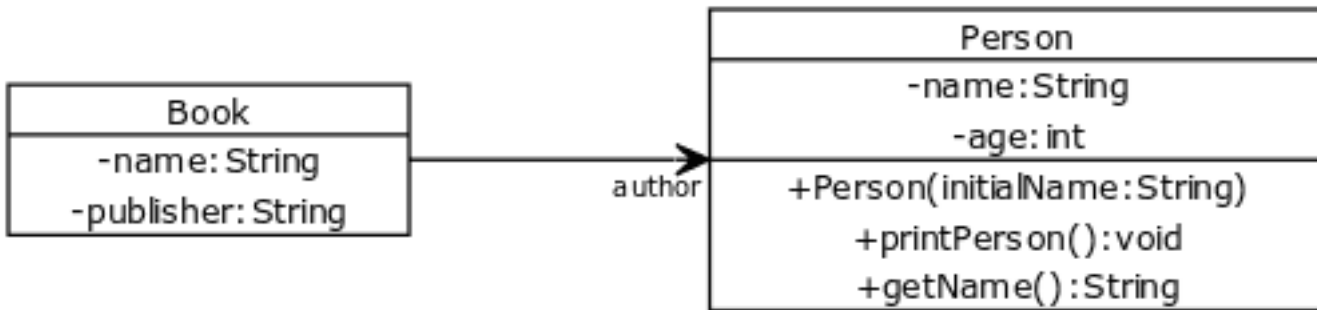
Person
-name: String -age: int
+Person(initialName: String) +printPerson(): void +getName(): String

Connections between classes

If we add a variable **author**, type of which is **Person**, in the source code the variable is declared like all other class variables.

```
public class Book {  
    private String name;  
    private String publisher;  
    private Person author;  
  
    // constructors and methods  
}
```

```
public class Book {  
    private String name;  
    private String publisher;  
  
    // constructors and methods  
}
```

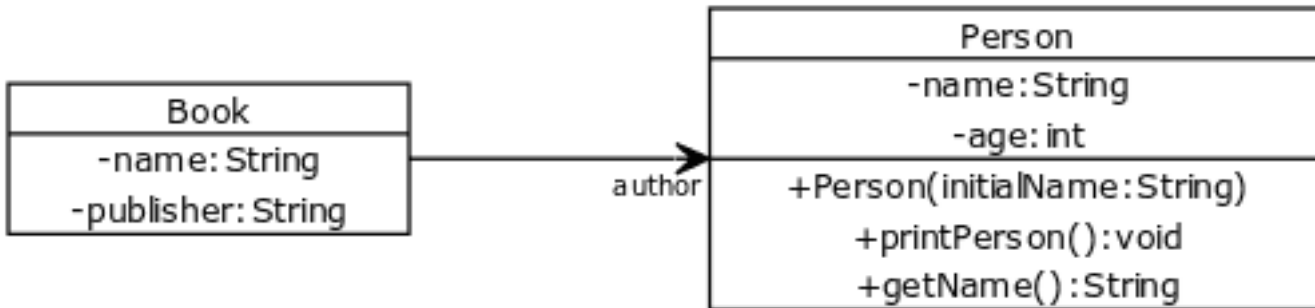


Connections between classes

In most UML class diagrams, **object references are better represented through relationships** rather than listing them as attributes inside the class box. However, if you want to explicitly show them as attributes, **it's not incorrect**, just less common in well-structured diagrams.

The arrow shows the direction of the connection.

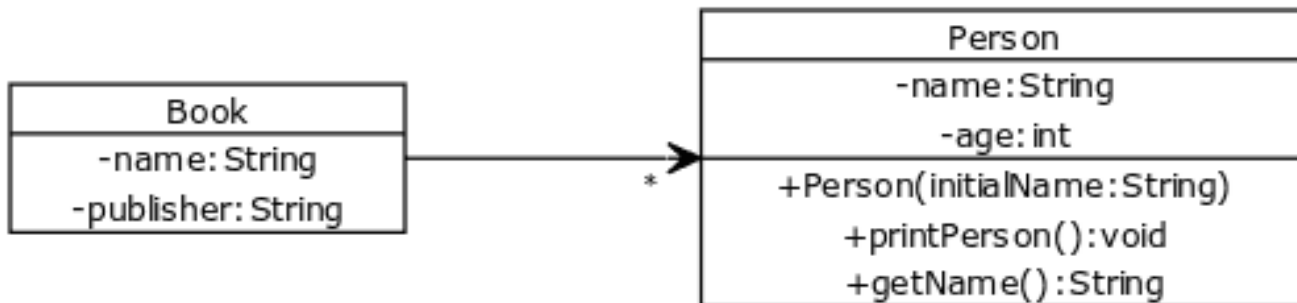
a Book knows its author but a Person does not know about books they are the author of



Connections between classes

```
public class Book {  
    private String name;  
    private String publisher;  
    private ArrayList<Person>  
authors;  
  
    // constructors and methods  
  
}
```

The star tells us that a book can have between 0 and unlimited number of authors.



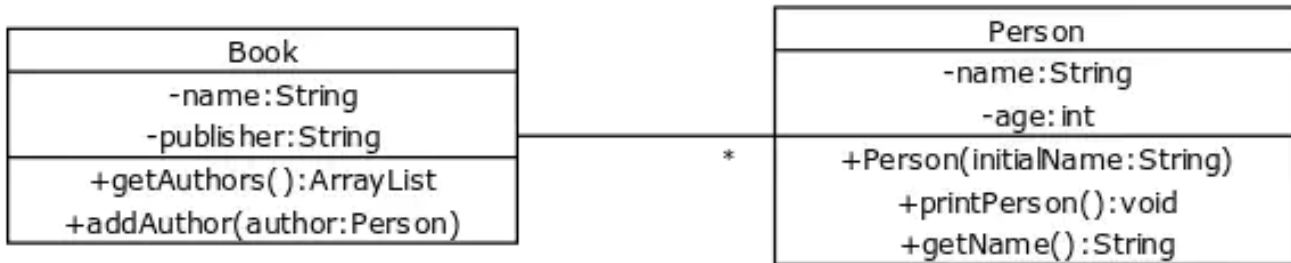
Connections between classes

If there is no arrowhead in a connection, both classes know about each other.

```
public class Book {  
    private String name;  
    private String publisher;  
    private ArrayList<Person> authors;  
  
    // ..  
}
```

```
public class Person {  
    private String name;  
    private int age;  
    private Book book;  
  
    // ...  
}
```

Person can only have one book.

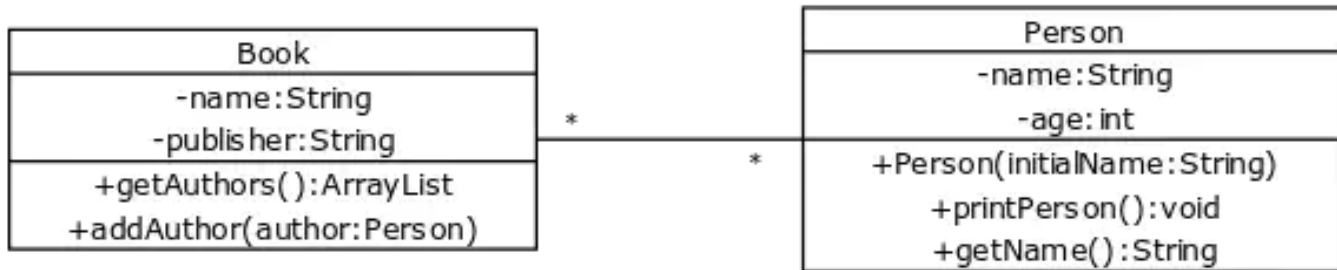


Connections between classes

If there is no arrowhead in a connection, both classes know about each other.

```
public class Book {  
    private String name;  
    private String publisher;  
    private ArrayList<Person> authors;  
  
    // ..  
}
```

```
import java.util.ArrayList;  
  
public class Person {  
    private String name;  
    private int age;  
    private ArrayList<Book> books;  
  
    // ...  
}
```

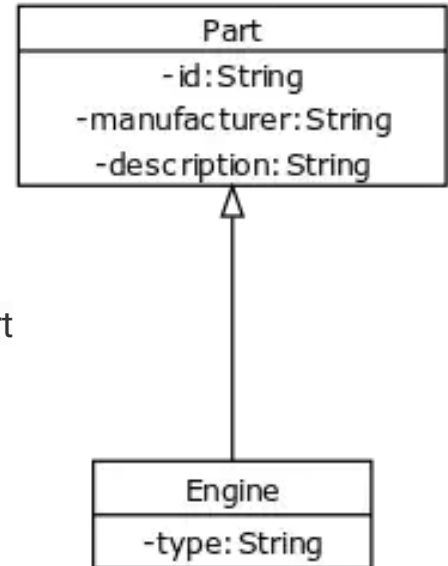


Describing inheritance

In a class diagram inheritance is described by an arrow with a triangle head.

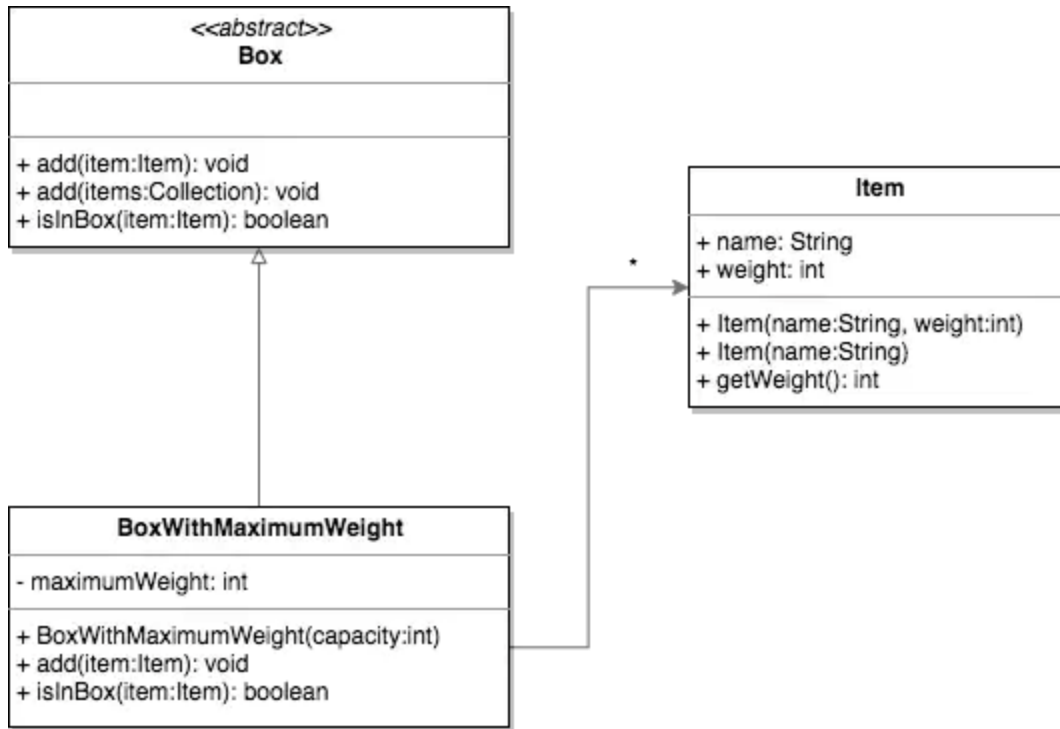
The triangle points to the class being inherited from.

the Engine inherits the class Part



Inheritance of abstract classes

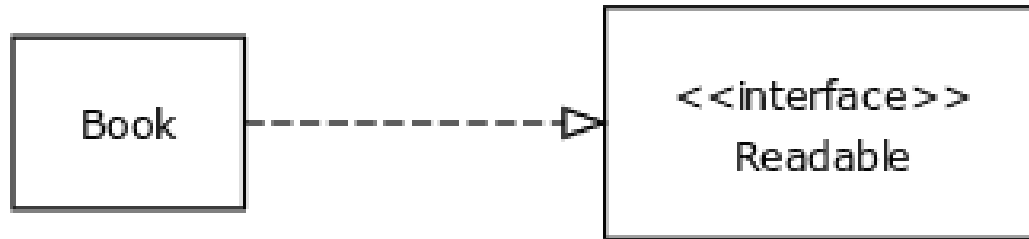
we add the description <<abstract>> above the name of the class



Describing interfaces

interfaces are written with <<interface>>

Implementing an interface is shown as a dashed arrow with a triangle arrowhead



Design patterns

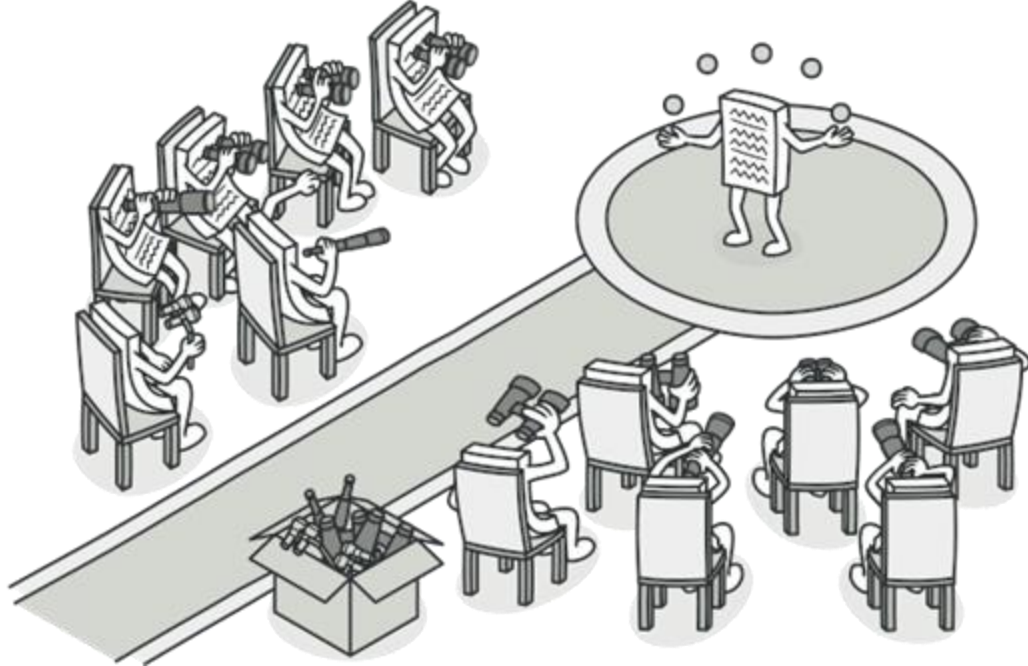
Observer





Observer

Observer is a **behavioral design pattern** that lets you define a **subscription** mechanism to **notify multiple objects** about any events that happen to the object they're observing.

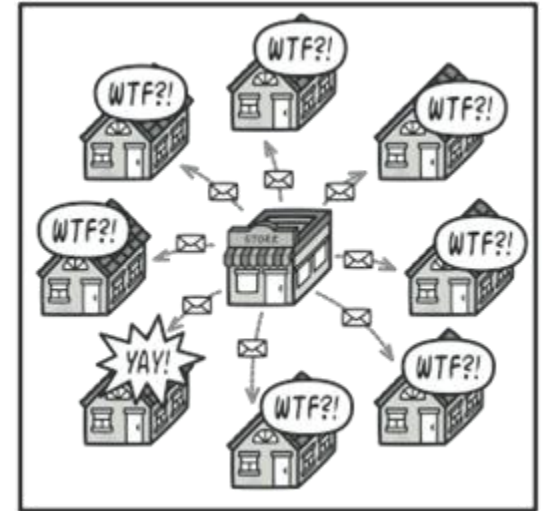
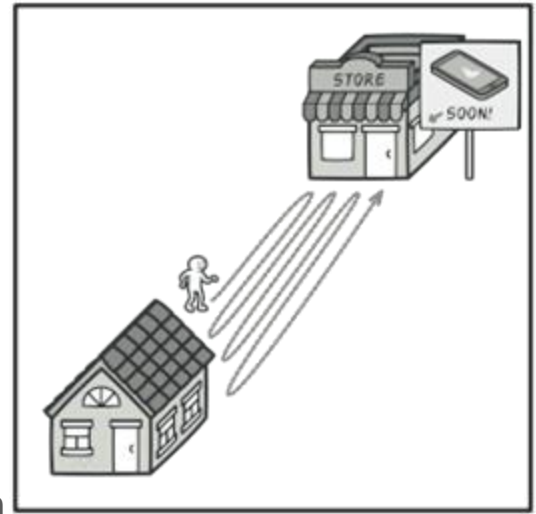


Problem

Imagine a Customer eagerly waiting for a new product (e.g., the latest iPhone) at a Store. The customer has two choices:

- **Constantly checking the store** – wasting time on pointless visits while the product is still en route.
- **The store mass-emailing all customers** – spamming uninterested customers to notify the few who care.

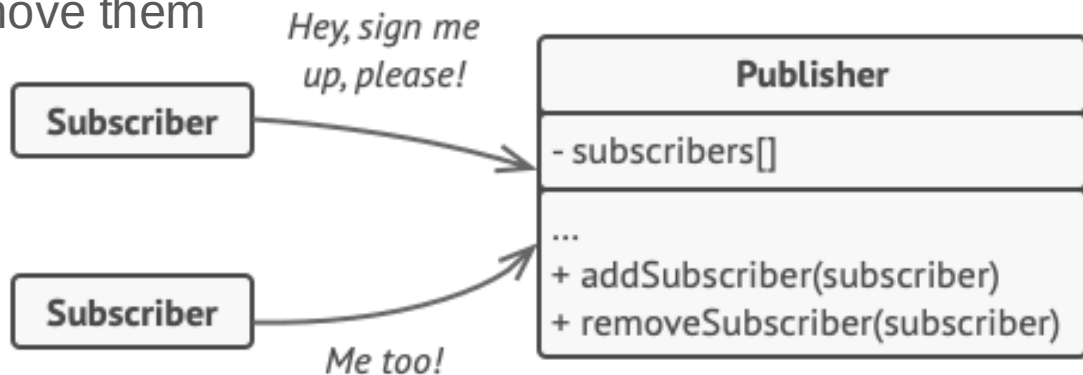
conflict: either the customer wastes time, or the store wastes resources.



Solution

In the Observer Pattern, the object with an interesting state is called the **Publisher** (or Subject). It notifies other objects, called **Subscribers**, about changes in its state.

- Subscribers can opt-in to receive updates and unsubscribe anytime.
- The Publisher maintains a list of subscribers and notifies them when a change occurs.
- Technically, this means storing an array of subscriber references and providing methods to add/remove them



Decoupling with a Common Interface

When an event occurs, the Publisher notifies all Subscribers by calling their notification method.

- **Challenge:**

- Real applications may have many different subscriber types, and the publisher shouldn't be tightly coupled to them. Some subscribers might not even be known in advance.

- **Solution:**

- All subscribers implement the same **interface**, ensuring the publisher interacts with them in a uniform way.
- This interface declares a notification method with parameters, allowing the publisher to pass relevant data without knowing the subscriber's internal details.
- Loose coupling = flexible & scalable design

Real-World Analogy

Subscribing to a newspaper or magazine works just like the Observer Pattern:

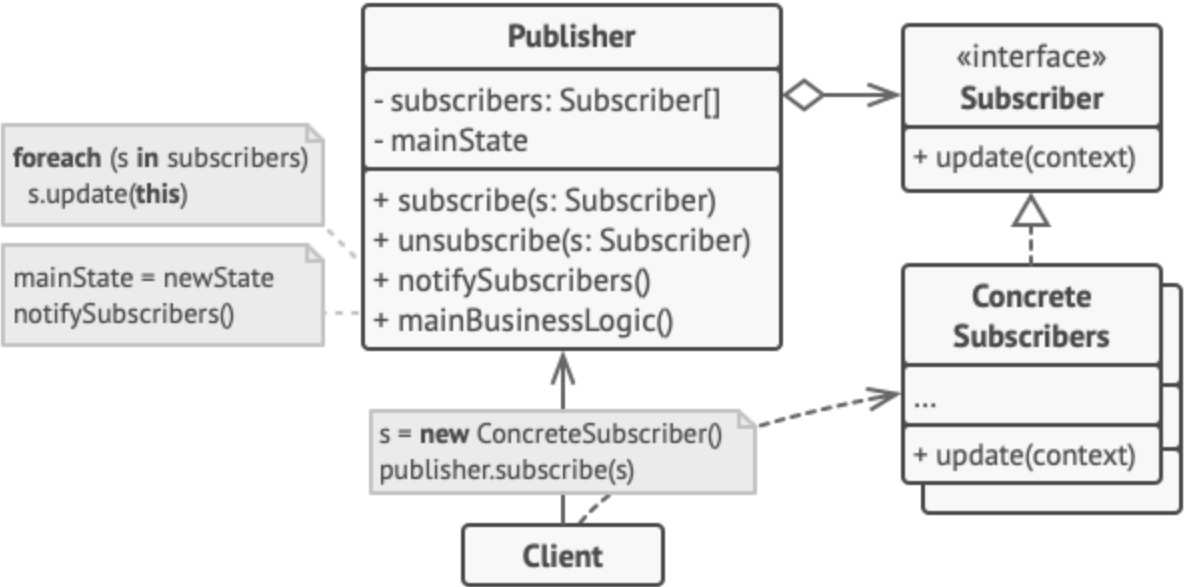
- Subscribers don't need to check the store for new issues - the Publisher delivers updates directly.
- The Publisher maintains a list of subscribers and knows what they're interested in.
- Subscribers can unsubscribe anytime, stopping further deliveries.
- efficient **push-based model**



Structure

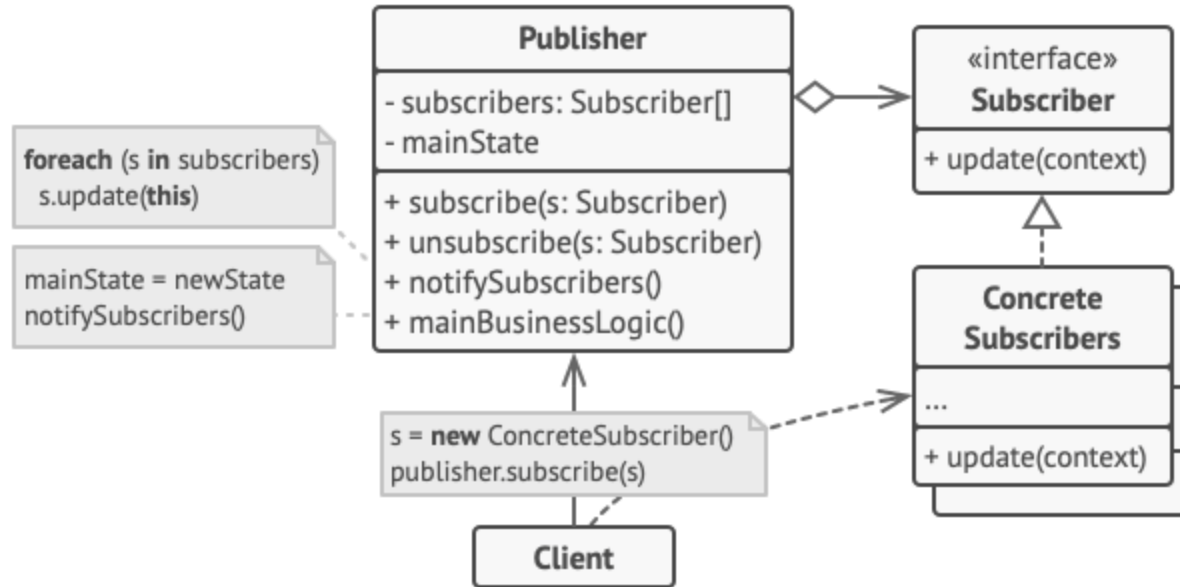
Publishers contain a subscription infrastructure that lets new subscribers join and current subscribers leave the list.

When a new event happens, the publisher goes over the subscription list and calls the notification method declared in the subscriber interface on each subscriber object.



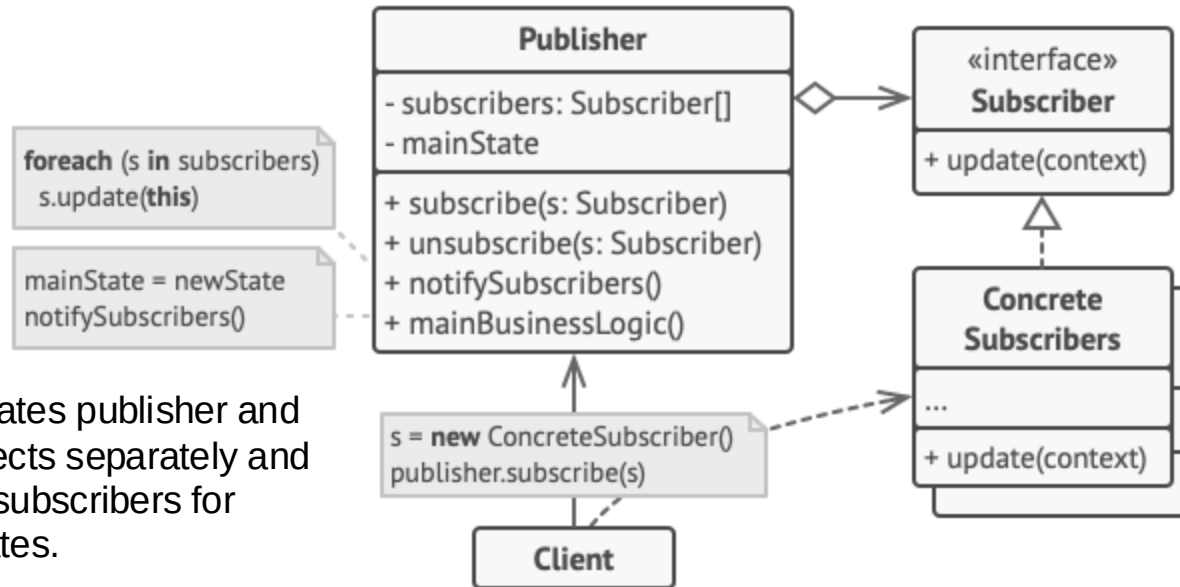
Structure

The Subscriber interface declares the notification interface. In most cases, it consists of a **single update method**. The method may have several parameters that let the publisher pass some event details along with the update.



Concrete Subscribers perform some actions in response to notifications issued by the publisher. All of these classes must implement the same interface so the publisher isn't coupled to concrete classes.

Structure



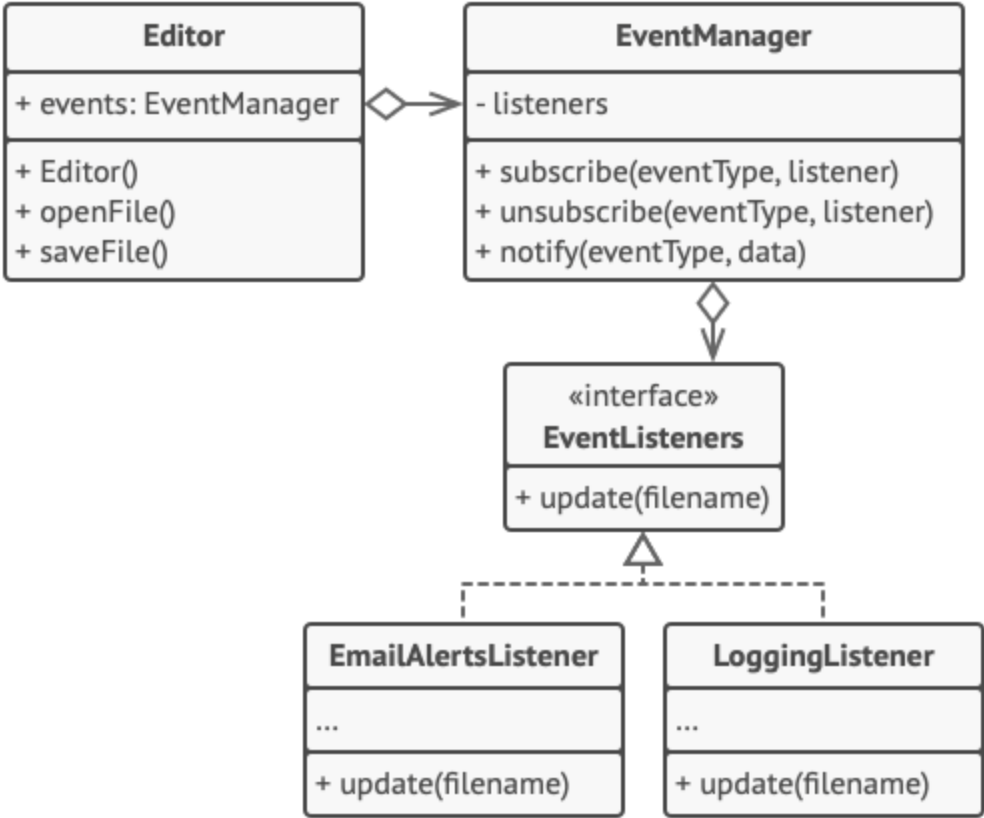
The **Client** creates publisher and subscriber objects separately and then registers subscribers for publisher updates.

Usually, subscribers need some **contextual information** to handle the update correctly. For this reason, publishers often pass some context data as arguments of the notification method. The publisher **can pass itself** as an argument, letting subscriber fetch any required data directly.

Structure

the Observer pattern lets the text editor object notify other service objects about changes in its state.

In this implementation, the editor class doesn't maintain the subscription list by itself. It delegates this job to the special helper object devoted to just that.



When to Use the Observer Pattern?

- **Dynamic Dependencies:**

- Use the Observer Pattern when changes in one object must trigger updates in other objects, but the exact set of dependent objects is unknown or changes dynamically.

- **GUI Event Handling:**

- custom buttons that allow clients to attach custom actions when clicked. The pattern enables buttons to notify subscribers (event listeners) without hardcoding dependencies.

- **Flexible Subscription Mechanism:**

- Any object implementing the Subscriber interface can receive notifications.
- Clients can attach custom behavior by subscribing their own classes.
- Subscribers can dynamically join or leave the list, allowing for temporary or case-specific event tracking.

How to Implement the Observer Pattern

- Step 1: Identify Roles
 - Break down your system into a Publisher (core logic) and Subscribers (dependent objects).
- Step 2: Define the Interfaces
 - Subscriber Interface: Declares an `update()` method for receiving notifications.
 - *Publisher Interface: Provides methods to add and remove subscribers.*
- Step 3: Implement Subscription Logic
 - Store the list of subscribers in an abstract base class (common behavior).
 - Concrete publishers extend this class and inherit subscription handling.
- Step 4: Create Concrete Publisher
 - When an event occurs, the Publisher iterates through the subscriber list and notifies them.
- Step 5: Implement Concrete Subscribers
 - Each Subscriber implements the `update()` method to react to changes.
 - Context data can be passed in the update method or retrieved from the publisher.
- Step 6: Client Setup
 - The client creates and registers Subscribers with the Publisher.

Pros and Cons

- + You can introduce new subscriber classes without having to change the publisher's code
- + You can establish relations between objects at runtime.
- Subscribers are notified in random order.

Observer vs mediator

Observer Pattern

- Establishes dynamic one-way communication between objects.
- Some objects (Subscribers) depend on another (Publisher) for updates.
- Used for event-driven systems, where changes in one object trigger actions in others.

Mediator Pattern:

- Eliminates direct dependencies among system components.
- Components communicate only through a central Mediator instead of knowing about each other.
- Used when you want to reduce coupling between interacting objects.

When Do They Overlap?

A Mediator can act as a Publisher, with components subscribing to its events—this makes Mediator look like Observer.

However, Mediator can also be implemented without Observer, by directly linking all components to the mediator.

Key Difference:

Observer: Objects communicate directly with multiple subscribers.

Mediator: Objects communicate indirectly through a central coordinator.